PROJECT: Scalable Web Application with Load Balancing

Services: EC2, ELB, Auto Scaling, RDS, S3, Route 53
Goal: Deploy a web app that scales based on load, with database and static file hosting.
Skills: VPC, security groups, EC2 launch templates, user data scripts.

Definition: A Scalable Web Application with Load Balancing is a system design where a web application can handle increasing user traffic automatically by adding or removing servers, while a load balancer distributes incoming requests evenly across those servers to ensure high performance and availability.
Scope of the project: Design and deploy a scalable web application using cloud services like EC2, Load Balancer, Auto Scaling, RDS, and S3.
Implement automatic scaling to handle varying user traffic efficiently without manual intervention.
Ensure high availability, fault tolerance, and optimized performance through load balancing and multi-AZ deployment.

Methodologies:

Method Used in the Project: The project uses the Auto Scaling method with Target Tracking Scaling Policy based on Application Load Balancer (ALB) Request Count per Target.

- Implemented load balancing using AWS Elastic Load Balancer to distribute incoming traffic across multiple EC2 instances.
- Used Auto Scaling Groups to dynamically add or remove EC2 instances based on CPU utilization and demand.
- Configured security groups, VPC, and subnets to ensure secure and efficient network communication.
- Integrated RDS for database management and S3 for storing and serving static files.

Services Used in this Project:

1. **Amazon EC2 (Elastic Compute Cloud)**
   • **Definition:** Amazon EC2 (Elastic Compute Cloud) is a service that provides scalable virtual servers in the cloud to run applications.
   • **Purpose**: Amazon EC2 (Elastic Compute Cloud) is a service that provides scalable virtual servers in the cloud to run applications.
   • **Role in this Project**: EC2 instances handle user requests from the Load Balancer and scale automatically using Auto Scaling when traffic increases.

2.ELB (Elastic Load Balancer)

- **Definition:** Elastic Load Balancer is an AWS service that automatically distributes incoming application traffic across multiple EC2 instances.
- **Purpose:** The ELB is used to distribute user requests evenly to the EC2 instances running the web application.
- **Role in this Project:** The ELB receives traffic from users and forwards it to healthy EC2 instances in the Auto Scaling Group to ensure high availability and performance.

3. Auto Scaling :

- **Definition:** Auto Scaling is an AWS service that automatically increases or decreases the number of EC2 instances based on application demand.
- **Purpose:** Auto Scaling is used to automatically adjust the number of EC2 instances in response to changes in user traffic.
- **Role in this Project:** Auto Scaling ensures the web application maintains performance and availability by launching new instances during high load and terminating them when load decreases.

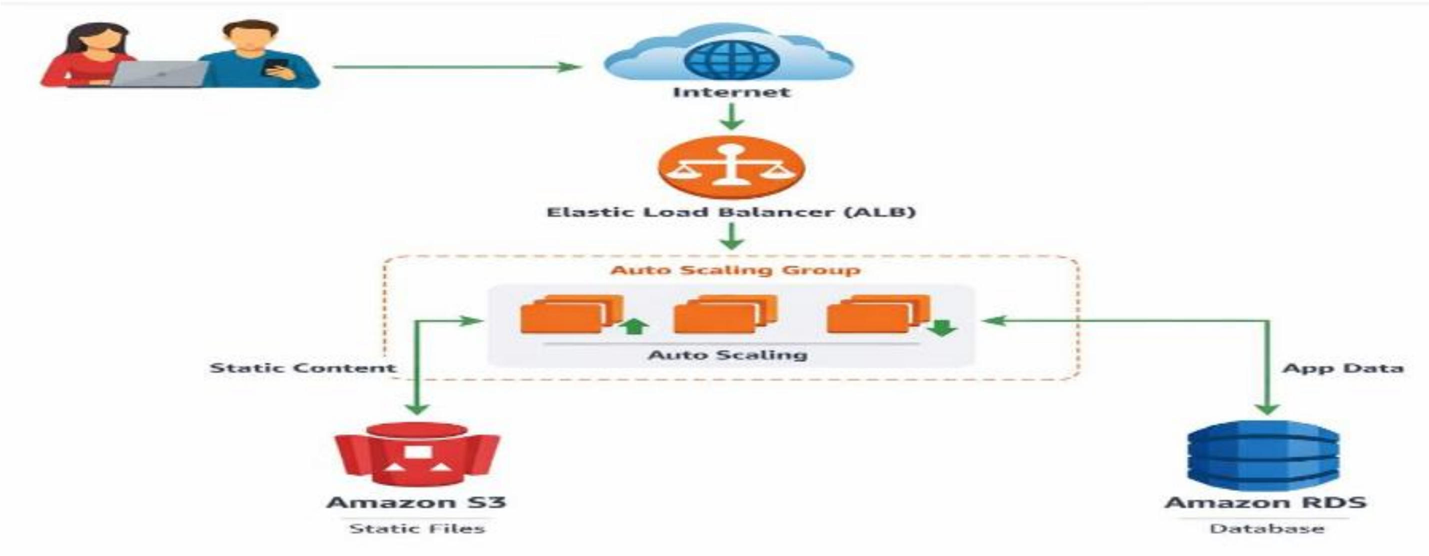
4. RDS (Relational Database Service)

- **Definition:** Amazon RDS is a managed AWS service used to create, operate, and manage relational databases in the cloud.
- **Purpose:** RDS is used to store and manage the web application’s database securely and reliably.
- **Role in this Project:** RDS is used to store and manage the web application’s database securely and reliably.

5. S3 (Simple Storage Service)

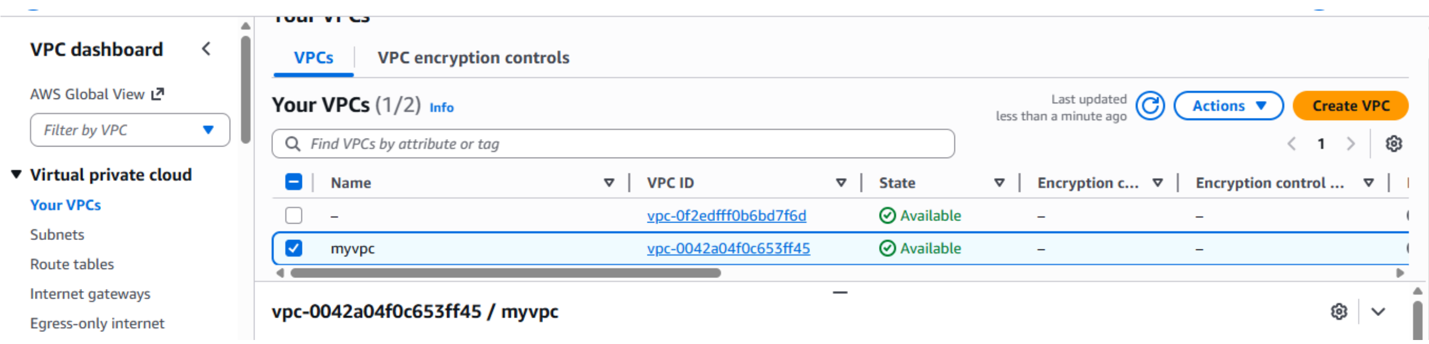
- **Definition:** Amazon S3 is a scalable cloud storage service used to store and retrieve files such as images, videos, and documents.
- **Purpose:** S3 is used to store the static files of the web application like images, CSS, and JavaScript.
- **Role in this Project:** S3 is used to store the static files of the web application like images, CSS, and JavaScript.

Architecture Diagram:

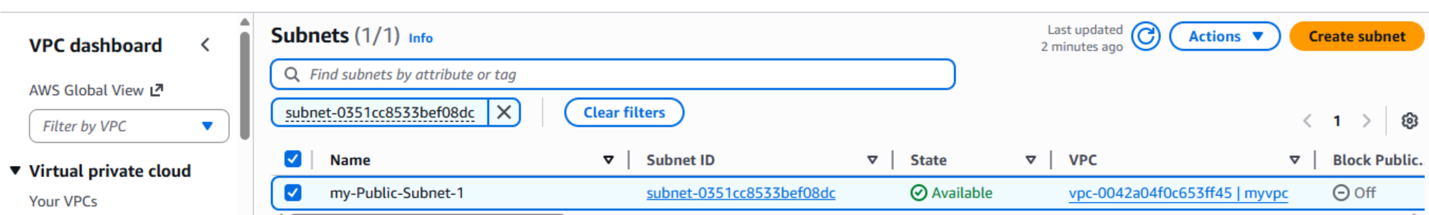


Implementation and Execution of Scalable Web Application with Load Balancing:

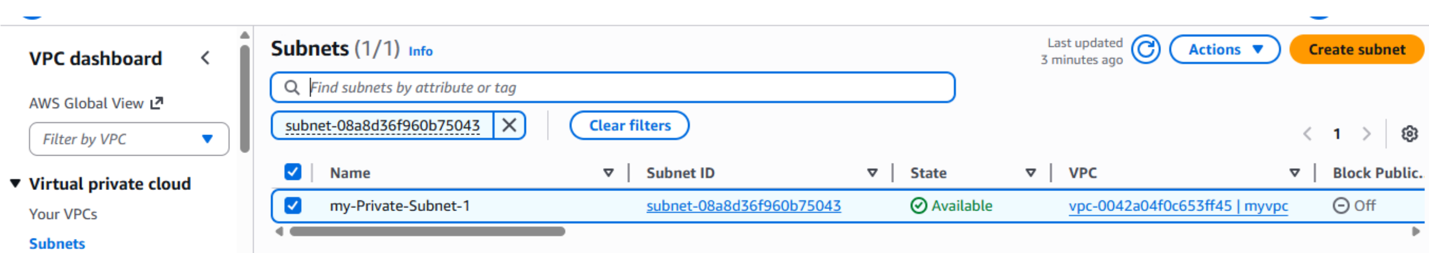
Step 1: Created VPC



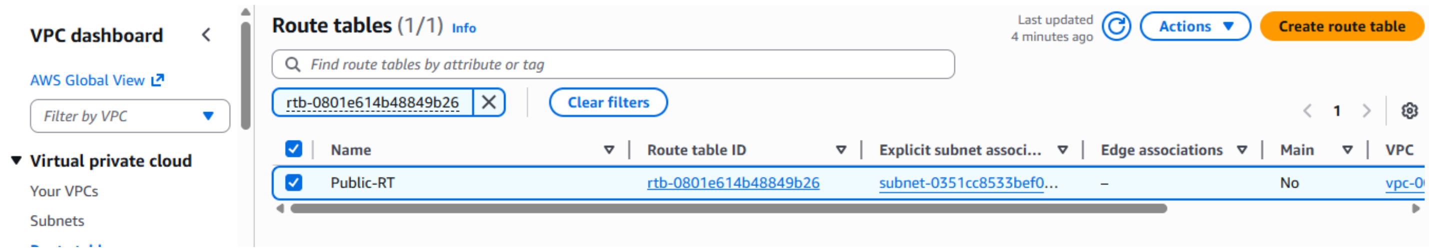
STEP 2: Create Public Subnet



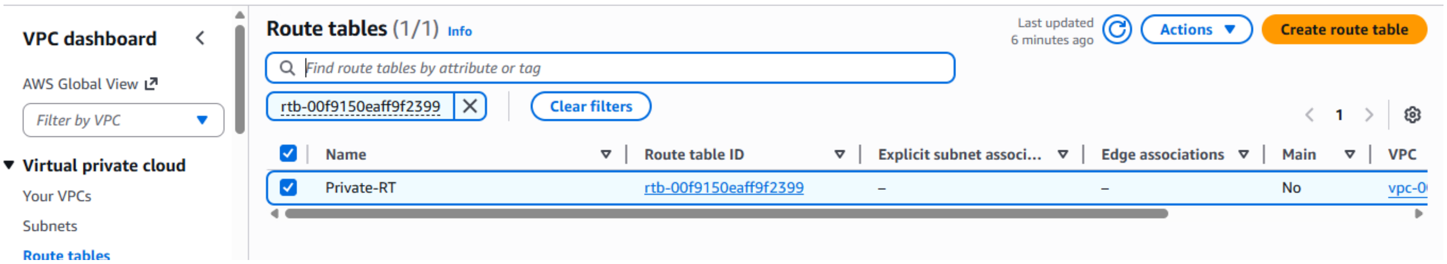
STEP 3: Create Private Subnet



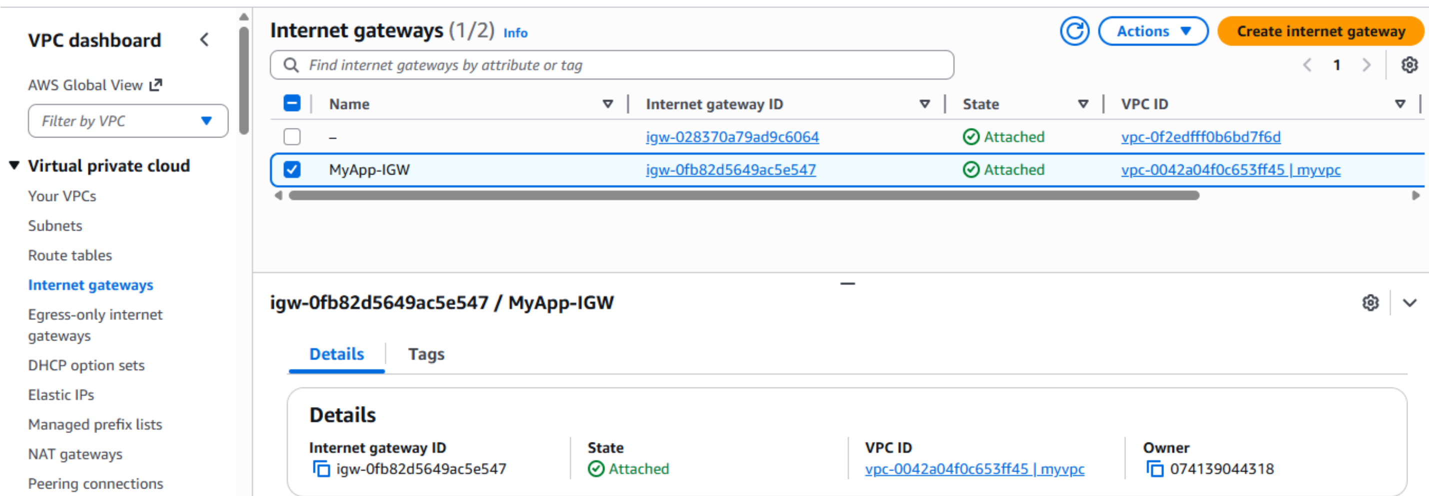
STEP 4: Create Public Route Table



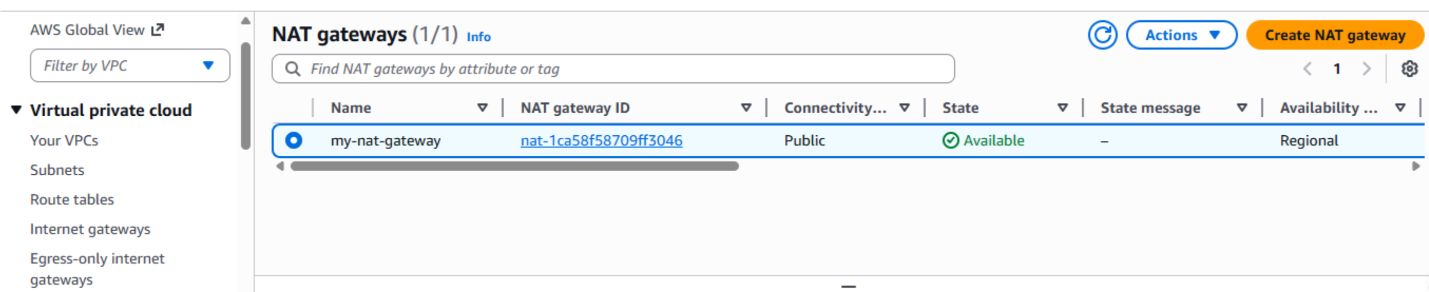
STEP 5: Create Private Route Table



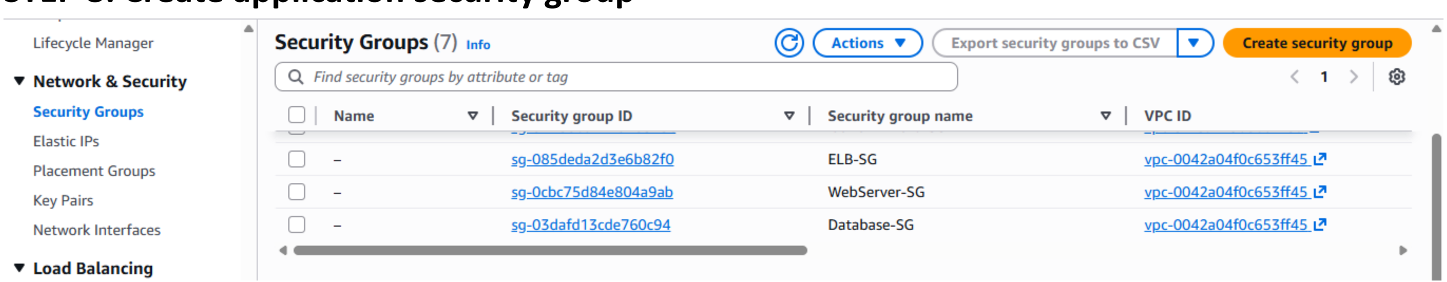
STEP 6: Created Internet Gateway and Attached to VPC.



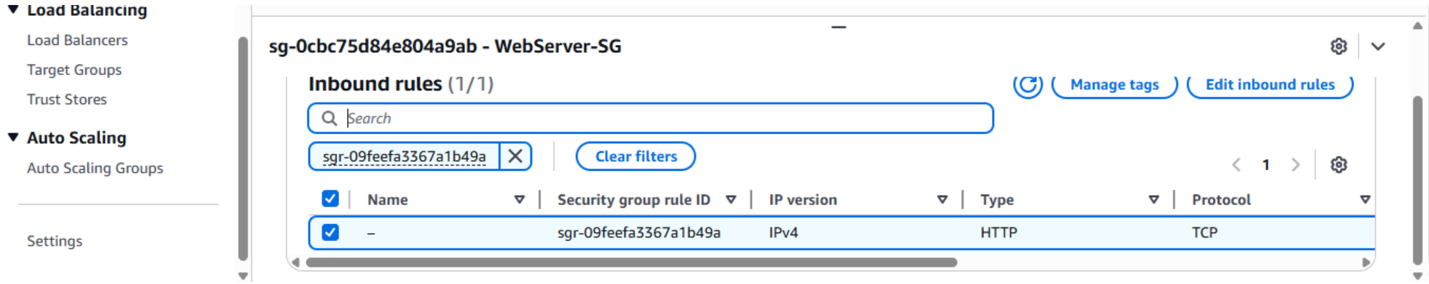
STEP 7: Create NAT Gateway



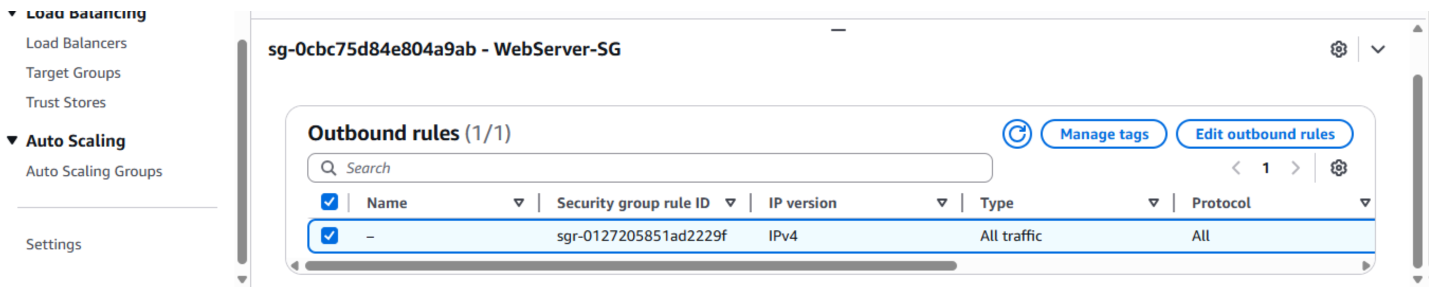
STEP 8: Create application security group



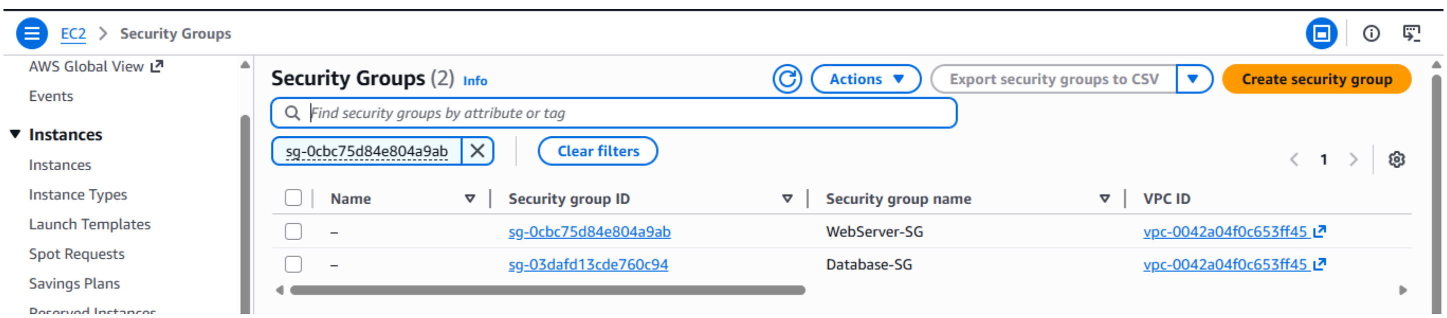
Inbound Rule:



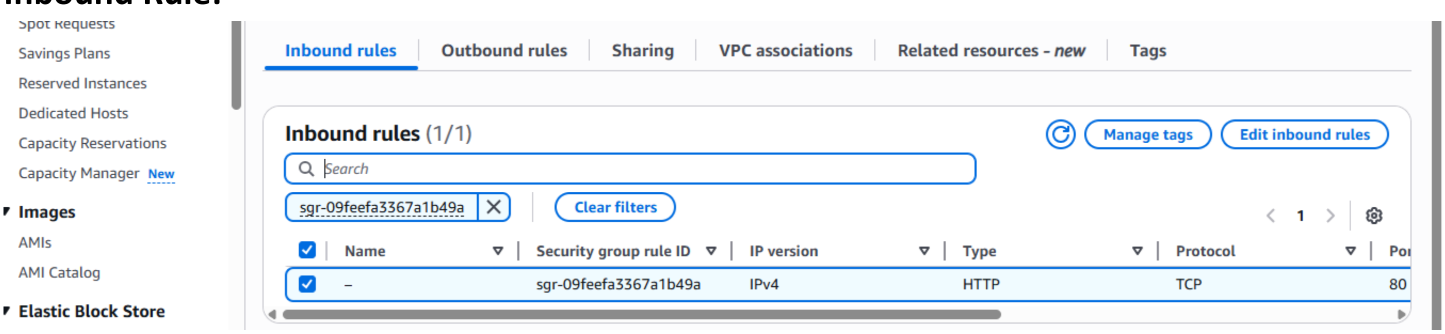
Outbound Rule:



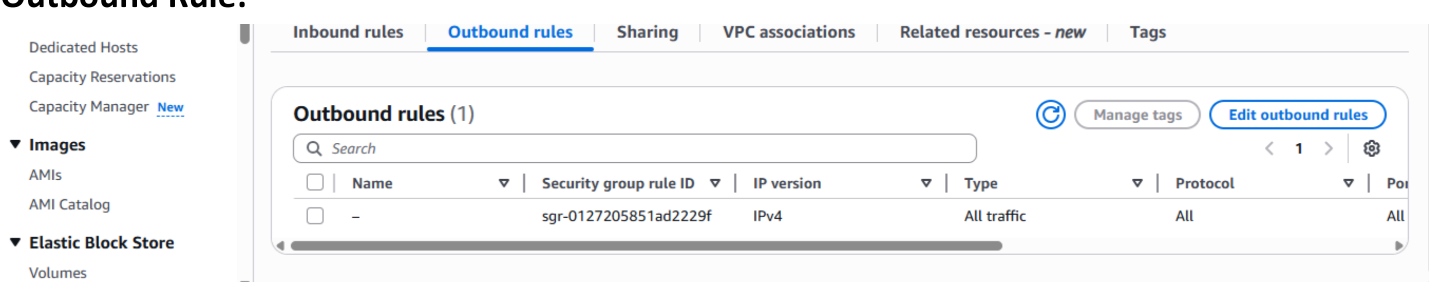
STEP 9: Create ec2 security group



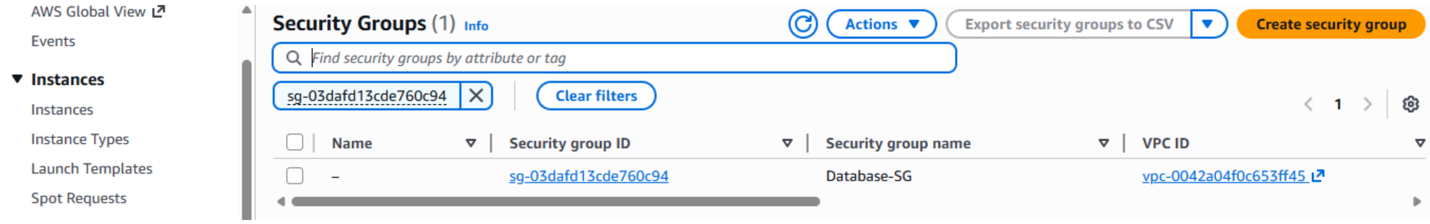
Inbound Rule:



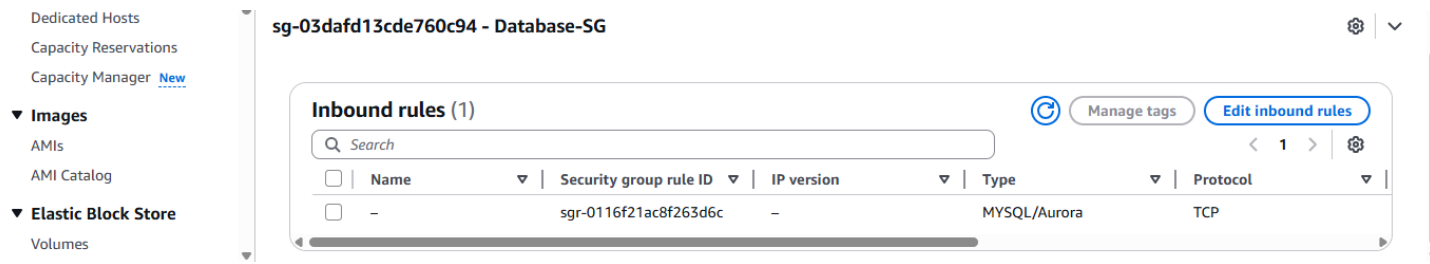
Outbound Rule:



STEP 10: Create Relational security group

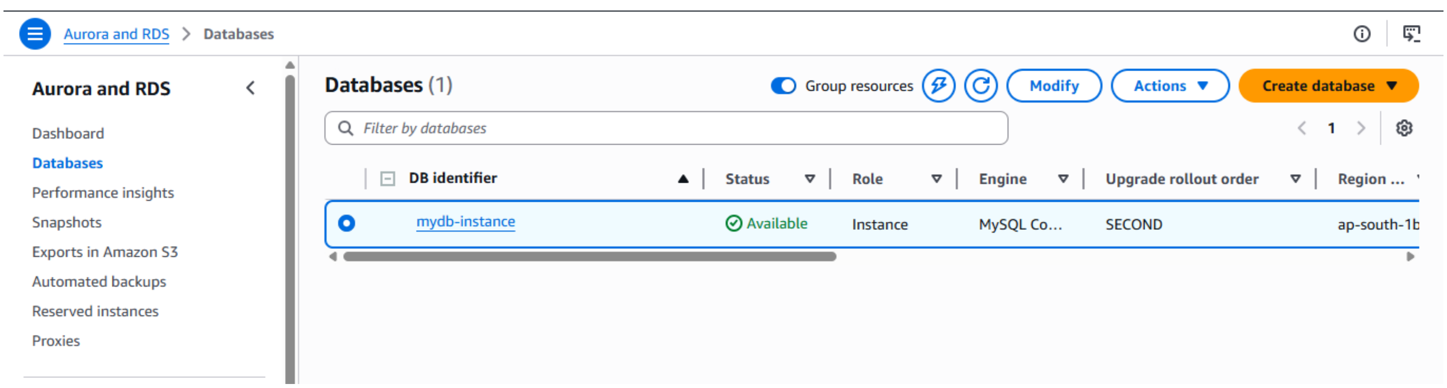


Inbound Rule:

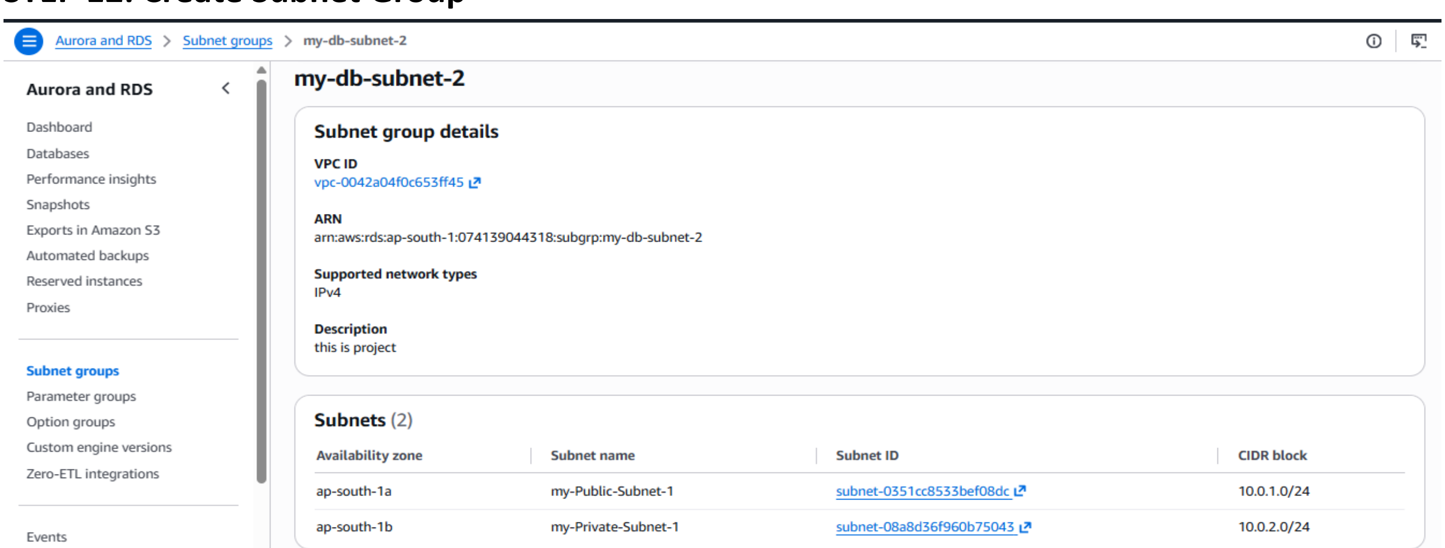


Outbound Rule:

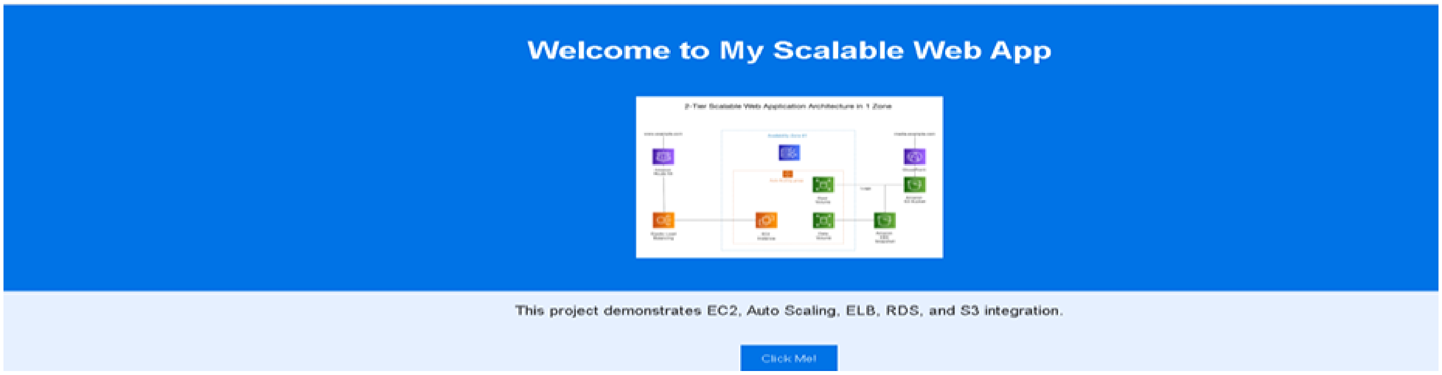
STEP 11: Create Database



STEP 12: Create Subnet Group

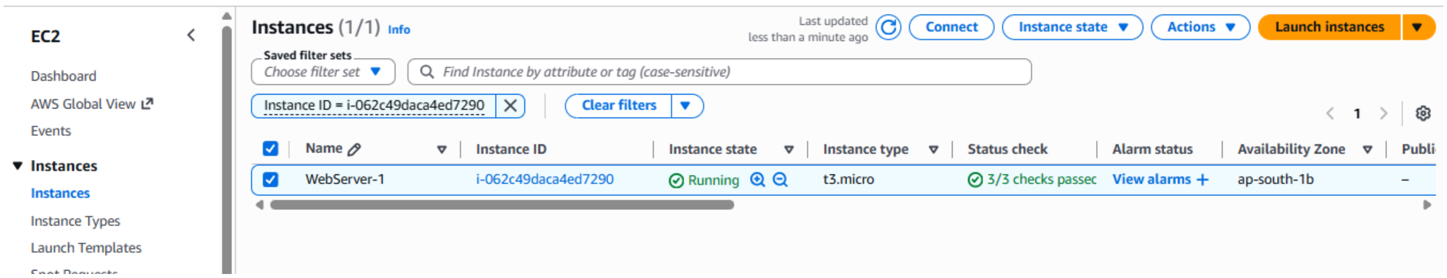


STEP 13: Final Output

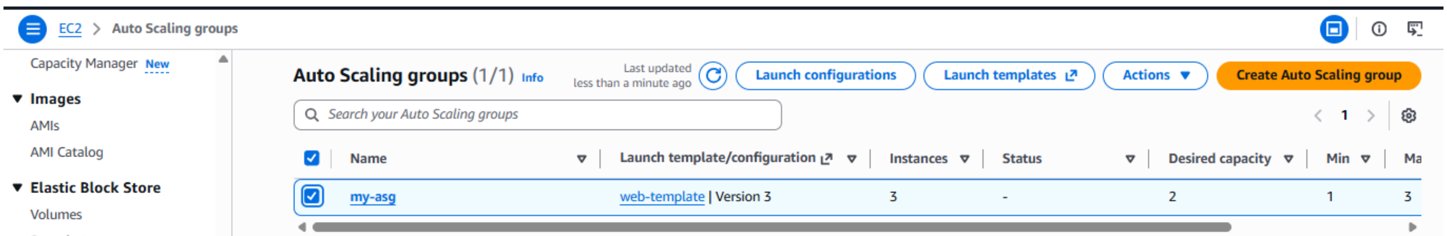


STEP 14: Before Load

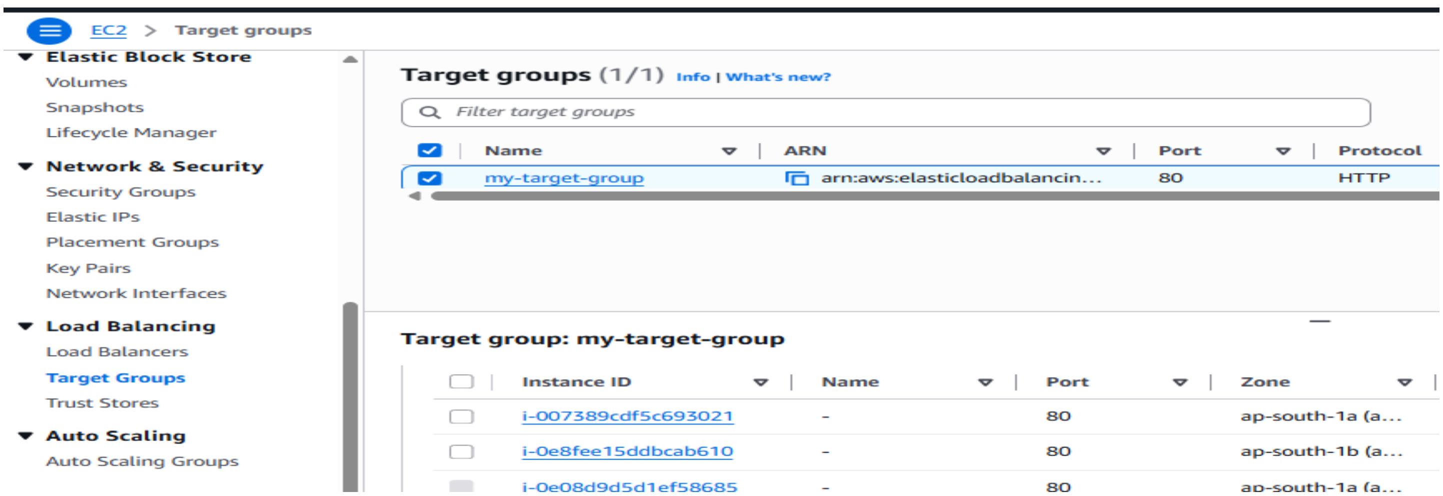
Instance:



Auto Scaling Group:

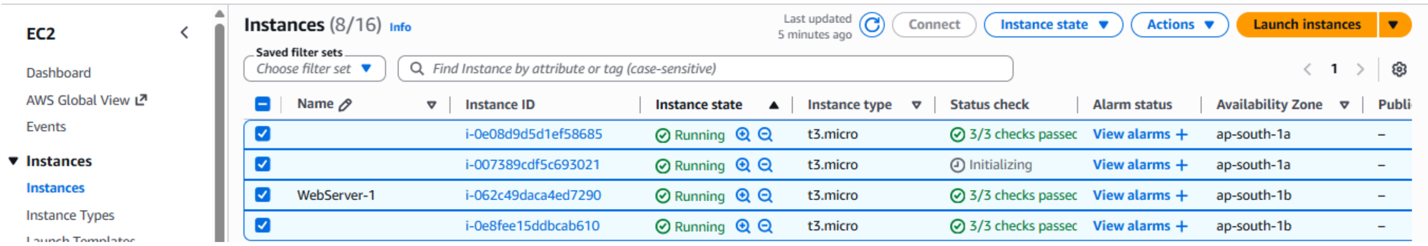


Target Group:

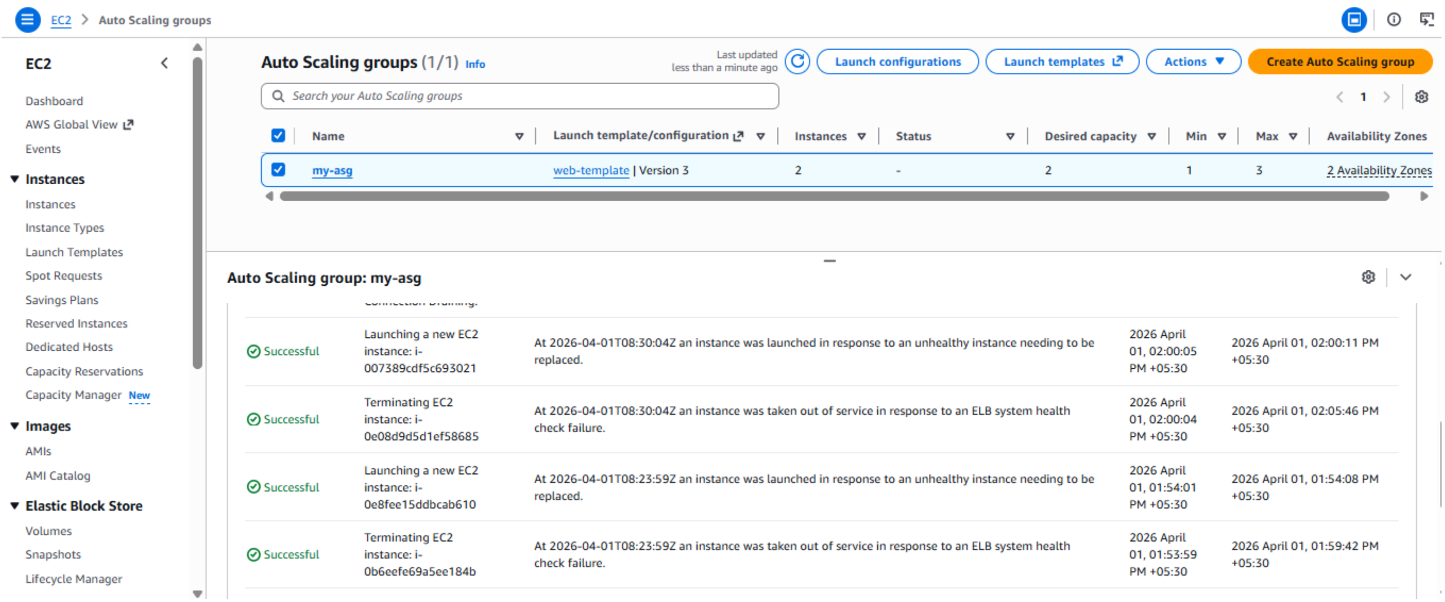


STEP 15: After Load

Instance:



Auto Scaling Group:



Troubleshooting:

Target Group Troubleshooting:

Issue: Health check path (e.g., /index.html) is incorrect or file missing.

Instance is stopped, terminated, or failing system status checks.

Solution:

- Start the EC2 instance (or let Auto Scaling recreate it) and ensure system status checks pass.
- Ensure correct path (e.g., /index.html) and create the file in /var/www/html/ on EC2.

Conclusion:

This project successfully demonstrated how to deploy a scalable and highly available web application using AWS cloud services. By integrating EC2, Application Load Balancer, Auto Scaling, Amazon S3, and Amazon RDS, the system was able to handle user traffic efficiently. The application automatically scaled out when the load increased and scaled in when the load decreased, ensuring performance and cost optimization. Overall, the project shows how cloud based architecture can provide reliability, flexibility, and efficient resource management for modern web applications.